IC253 Assignment 2 Report

Daksh Rathi
B25132

1 Introduction

This report presents the implementation of two linked-data-structure based systems. Task 1 implements a gaming leaderboard using a Skip List, and Task 2 implements a messenger system using a Doubly Linked List with time-bound deletion. The report mainly focuses on the implementation approach, data structures used, and time and space complexity.

2 Task 1: Gaming Leaderboard using Skip List

2.1 Objective

The objective of this task is to maintain a leaderboard in which players are ranked by score, and for equal scores, by earlier timestamp. The system supports insertion, deletion, updating, searching, rank retrieval, top-k display, and leaderboard visualization.

2.2 Approach

The leaderboard is implemented using a Skip List so that player records remain stored in sorted order. The implementation uses the comparison function `isBefore()` to decide ranking order exactly according to score, timestamp, and player ID.

Each player is stored inside a `SkipListNode` containing one `PlayerData` object, the node level, and the forward pointer array `next[maxLevel + 1]`. The full structure is managed by the `SkipList` class, which maintains the sentinel node `Head` and the highest active level `currentLevel`.

The height of each newly inserted node is generated by `randomLevel()`. During `insertPlayer()`, traversal starts from the highest active level and moves downward. The implementation uses the dynamically allocated pointer `modifyNode` to remember the required predecessor links during insertion. The `deletePlayer()` and `searchPlayer()` operations also use top-down level traversal. The `updateScore()` operation is implemented as delete followed by reinsert, because changing the score can change the player's position in the leaderboard.

2.3 Algorithm Summary

The main steps of the Task 1 implementation are:

1. Initialize the `SkipList` with sentinel node `Head` and level tracker `currentLevel`.
2. Use `randomLevel()` to determine the height of each new `SkipListNode`.
3. Use `isBefore()` to compare two `PlayerData` records.
4. Perform `insertPlayer()` by traversing level by level and storing predecessor pointers in `modifyNode`.
5. Perform `searchPlayer()` by moving forward at each level and descending when needed.
6. Perform `deletePlayer()` by locating the target node and reconnecting links level-wise.
7. Perform `updateScore()` by removing the old record and reinserting the updated record.
8. Use `getRank()`, `getTopK()`, `displayLeaderboard()`, and `displaySkipListStructure()` for final output.

2.4 Data Structures

The following structures are used in the implementation:

- `PlayerData` : stores `playerID`, `name`, `score`, and `timestamp`.
- `SkipListNode` : stores one `PlayerData` object, `nodeLevel`, and `next[maxLevel + 1]`.
- `SkipList` : manages `Head`, `currentLevel`, and all leaderboard operations.
- `modifyNode` : dynamically allocated pointer used during `insertPlayer()` to remember predecessor links.

2.5 Time and Space Complexity Analysis

Let n denote the number of players.

Worst case occurs when the skip list degenerates into a simple linked list, that is, when most nodes remain only at level 0.

Function	Expected Time	Worst Case Time	Space	Remarks
randomLevel()	$O(1)$	$O(\text{maxLevel})$	$O(1)$	maxLevel fixed at 16
isBefore()	$O(1)$	$O(1)$	$O(1)$	compares score, timestamp, and player ID
insertPlayer()	$O(\log n)$	$O(n)$	$O(1)$	uses dynamically allocated `modifyNode`; worst case when skip list degenerates
deletePlayer()	$O(\log n)$	$O(n)$	$O(1)$	level-wise search and relinking
updateScore()	$O(\log n)$	$O(n)$	$O(1)$	implemented as delete + insert
searchPlayer()	$O(\log n)$	$O(n)$	$O(1)$	top-down level traversal
getRank()	$O(n)$	$O(n)$	$O(1)$	scans bottom level sequentially
getTopK(k)	$O(k)$	$O(n)$	$O(1)$	prints first k players from level 0
displayLeaderboard()	$O(n)$	$O(n)$	$O(1)$	full traversal of level 0
displaySkipListStructure()	$O(n)$	$O(n)$	$O(1)$	levels bounded by fixed `maxLevel`

2.6 Test Cases

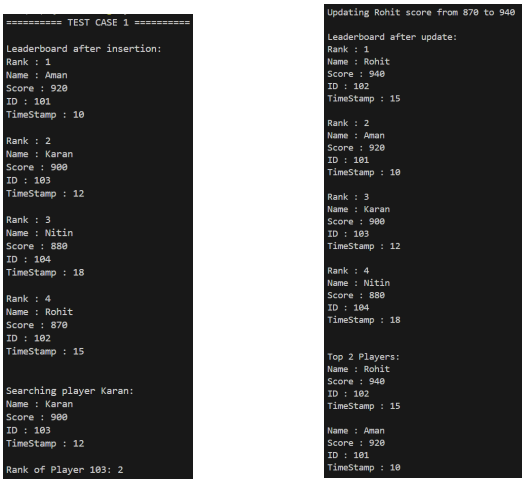

Figure 1: Task 1 – Test Case 1 Output

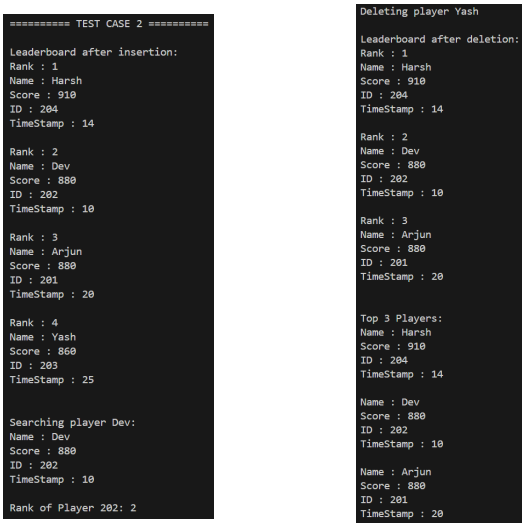

Figure 2: Task 1 – Test Case 2 Output

3 Task 2: Time-Bound Message Deletion using Doubly Linked List

3.1 Objective

The objective of this task is to implement a messenger system with ordered chat storage, delete support, and undo functionality. The operation `deleteForEveryone()` is allowed only within 120 seconds of sending.

3.2 Approach

The messenger system is implemented through the `MessengerSystem` class. The chat itself is stored as a doubly linked list using `MessageNode` nodes, while undo support is provided through a stack implemented using `StackNode` and the `Stack` class.

Each chat message is stored in the `Message` structure containing `msgID`, `senderID`, `text`, `timestamp`, and `isDeleted`. The complete chat is managed using `msgHead` and `msgTail`, while deleted messages are tracked using the stack object `DeletedMessage`.

The operation `sendMessage()` inserts a new message at the tail. The function `findNode()` is used to locate a message by ID. Then either `deleteForMe()` or `deleteForEveryone()` is applied. Every successful delete pushes that message node onto the stack using `push()`, and `undoLastDelete()` restores the most recent deletion using `pop()`.

3.3 Algorithm Summary

The main steps of the Task 2 implementation are:

1. Store each chat record in a `Message` object.
2. Maintain the chat sequence using `MessageNode` nodes linked through `msgHead` and `msgTail` present in the `MessengerSystem`.
3. Insert each new message at the tail using `sendMessage()`.

4. Use `findNode()` to locate a message by `msgID`.
5. Apply `deleteForMe()` or `deleteForEveryone()` depending on the requested action.
6. Record each successful delete using stack `push()` inside `DeletedMessage`.
7. Restore the most recent deletion using `undoLastDelete()`, `pop()`, and the stored `MessageNode*`.
8. Print the current chat using `displayChat()`.

3.4 Data Structures

The following structures are used in the implementation:

- `Message` : stores `msgID`, `senderID`, `text`, `timestamp`, and `isDeleted`.
- `MessageNode` : stores one `Message` object along with `prev` and `next` pointers.
- `StackNode` : stores a pointer to `MessageNode` and a pointer to the next `StackNode`.
- `Stack` : implements undo support through `push()`, `pop()`, and `isEmpty()`.
- `MessengerSystem` : manages `msgHead`, `msgTail`, `DeletedMessage`, and all message operations.

3.5 Time and Space Complexity Analysis

Let n be the number of messages.

Function	**Expected Time**	**Worst Case Time**	**Space**	**Remarks**
sendMessage()	$O(1)$	$O(1)$	$O(1)$	inserts at `msgTail`
findNode()	$O(n)$	$O(n)$	$O(1)$	linear traversal from `msgHead`
deleteForEveryone()	$O(n)$	$O(n)$	$O(1)$	search + 120-second time check
deleteForMe()	$O(n)$	$O(n)$	$O(1)$	search + delete flag update
displayChat()	$O(n)$	$O(n)$	$O(1)$	prints chat in sending order
undoLastDelete()	$O(1)$	$O(1)$	$O(1)$	uses `DeletedMessage.pop()` and restores `isDeleted`
push()	$O(1)$	$O(1)$	$O(1)$	inserts at stack head
pop()	$O(1)$	$O(1)$	$O(1)$	removes stack head
isEmpty()	$O(1)$	$O(1)$	$O(1)$	checks whether undo is possible

Total memory usage of the complete messenger system grows linearly with the number of messages stored.

$$O(n)$$

3.6 Test Cases

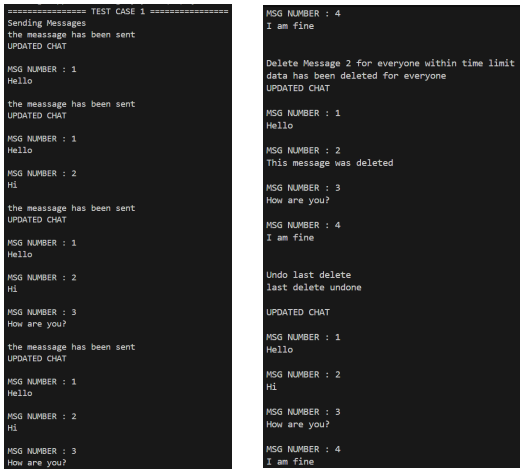

Figure 3: Task 2 – Test Case 1 Output

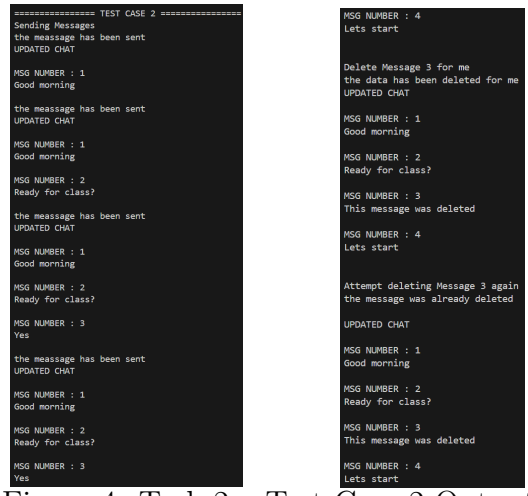

Figure 4: Task 2 – Test Case 2 Output

4 GitHub Repository

The complete source code and associated test cases for both Task 1 and Task 2 are available in the following repository:

- `https://github.com/dakshrathi-india/DSA-ASSIGNMENT-2`